BRAVO PRODUCTION FINDINGS · 03 OF 05 · OBSERVABILITY

What Bravo can actually see

> *Can Bravo ask "which loans are stuck in survey?" Are its activity logs thin and untagged? Can it tell which upstream service is failing, or only that upstreams are unreliable? Is there proper monitoring? And are logs really the only place anything is kept?*

0	**266,767**	**38**	**2.41M**
CAMUNDA TRACES. THE LAYER THAT RUNS THE WORKFLOW PRODUCES NONE.	404S A WEEK FROM ONE ENDPOINT, REACHING 69% OF SURVEYOR SESSIONS	MONITORS MATCH `BPM`. **NONE** OF THEM WATCHES A LOAN'S PROGRESS.	FRONT-END ERRORS IN 7 DAYS. THE MONITORING WAS THERE ALL ALONG, AND NOBODY READ IT.

Audience Production, on-call, Squad LOS, Squad S&U **Sources** Datadog us5 — APM, logs, monitors, RUM **Read** 2026-09-10

Most of the stated premises turned out to be wrong — in both directions

Bravo can ask "which loans are stuck in survey?"	● TRUE IN SQL ONLY	False everywhere an on-call engineer would actually look. `surveyor_assignment.assignment_status` is a `WHERE` clause, and that is a genuine Bravo advantage over LORA. But **no dashboard, monitor, SLO or APM view asks that question**, the Camunda runtime tables are deleted after **90 days**, and `Application.lastStage` is `@Transient` — the current stage is never saved. The capability exists. The practice does not.
...so stuck loans cannot be grouped across the whole book	● REFUTED	By a different route than we expected. Camunda's engine log lines carry `@activityName`, `@activityId` and `@processDefinitionId`. So failures **can** be ranked per BPMN step today: `One Obligor Check` had 5,583 errors in 48 hours, `PD Model Check NTB` 1,394. **Nobody was doing it.**
Activity logs are thin and untagged	● REFUTED FOR THE ENGINE	Confirmed for everything else — and **the real defect is a different one**. The 5% or so of lines that come from the Camunda job executor are richly tagged, right down to the *version* of the process definition. The other 95% are Spring stack traces with no workflow context at all. And **no log line carries the application or lead id**, so you can group failures by step but you cannot connect a failure to a loan. In LORA, `@WorkflowID` *is* the loan id.
We can tell which upstream service is failing	● TRUE, AND BETTER THAN LORA	25 upstream services can be told apart cleanly from `okhttp.request` traces, using `@peer.hostname`, with status codes. Because there is one Feign client per upstream, **you get this for free from the design**. LORA puts about 300 proxies behind a single route and cannot do this at all without an unreleased PR.

And the main thesis is upside down

Upstream BFI services are unreliable	● REFUTED	20,802 outbound errors in 7 days, and **not one 5xx in the top 20 rows**. Every upstream failure we can attribute is a 4xx: `401` from IAM (8,533), `404` from repeat-order (5,070), `400` from scheduling (3,508). **Those are contract and authentication defects, not outages.**
Bravo has no proper monitoring	● WRONG ON THE COUNT, RIGHT ON THE BPM LEVEL	38 monitors match `bpm`. Every one watches a container, a JVM, an HTTP endpoint or a queue. **None watches a running workflow, a Camunda incident, the job-executor backlog, or `application_error_tracking`.** Eight are in `Alert` right now; six read `No Data` and cannot fire.
Logs are the only place anything is kept	● REFUTED, AND THE REVERSE IS TRUE	What Bravo keeps permanently is in **PostgreSQL** — 238 entities, an `application_status_log` written by a database trigger, each reprocess attempt chained as a row. Logs are the *least* permanent layer here, and it is Camunda's own history that expires. **This is the opposite of LORA.**
Bravo's front ends are unmonitored	● REFUTED — AND THE LARGEST SIGNAL IS DISMISSED HERE	All four iOS consumers send browser monitoring at `ALL`. The Surveyor Platform produces **1,962,909 errors in 7 days**, of which **748,686 are HTTP 404s**; the biggest single one hits **54,350 of 79,090 sessions. No alert can see it, because every success-rate monitor only counts codes starting with 5.**

> **Bravo's problem is not that the data is missing. Most of it is there, and in two places it is better than LORA's.**
>
> The problem is that every alert Bravo owns watches the container and the 5xx — so a system that fails with 4xx, or by parking a workflow, cannot set one off.

“Which loans are stuck in survey?” — answerable in SQL, invisible in monitoring

This is the question LORA cannot answer in Temporal at all: no search attributes, no query handlers. Bravo can, and the reason is architectural. Business state lives in **one relational data model**, so *“which applications are sitting in surveyor assignment, and for how long?”* is a `WHERE` clause over `surveyor_assignment.assignment_status` (25 values) joined to `application`. **That advantage is real, and this document does not diminish it.** Three things limit it.

THE CURRENT STAGE IS NEVER SAVED

`Application.lastStage` is `@Transient`. Where a loan has got to is recovered from Camunda’s runtime tables instead. So the SQL answer is *“stuck in this assignment status”*, not *“stuck at this BPMN step”* — and the two differ whenever the step is one the assignment tables do not model.

CAMUNDA DELETES ITS HISTORY AFTER 90 DAYS

`historyTimeToLive: P90D`, with history level `full`. `act_hi_actinst` holds roughly **50 million rows in each 90-day window**. After that window, the only thing that can say what happened is `application_status_log`, written by a trigger — and that is a ladder of statuses, not a trace of steps.

NOBODY HAS WRITTEN THE QUERY DOWN

No dashboard widget, no saved query, no monitor and no SLO anywhere in the Datadog org asks “which applications have been stuck at stage X for more than N hours?” The question can be answered by somebody who opens a psql session and knows the schema. **It cannot be answered by whoever is on call at 02:00.**

> **Grouping stuck loans across the whole book does work today, and nobody was doing it.**
>
> That is the part of this finding that was simply wrong — see the next two sections.

The layer that runs the workflow produces no traces at all

OPERATION_NAME	TRACES / 7 D	WHAT IT IS
repository.operation	1,412,356	Database queries via JPA and Hibernate
spring.handler	990,256	REST controllers
okhttp.request	335,731	Calls out to upstream services
servlet.request	335,010	Incoming HTTP
http.request	84,743	Other outbound client calls
scheduled.call	1,193	`@Scheduled` and ShedLock jobs
jakarta_rs.request	75	—
Camunda traces of any kind	**0**	**no workflow instance, no service task, no `JavaDelegate`, no job execution**

Those seven names cover every trace `prod-ms-bpm` produced in 7 days. Grouping `spring.handler` by resource confirms it: the top twenty are all controllers, and not one is an `*Activity` bean.

THIS IS STRUCTURAL, NOT A SAMPLING OVERSIGHT

All **483 service tasks** are wired up through `camunda:delegateExpression` and run inside job-executor threads. Those threads are **not started by an HTTP request**, so the Java agent has no trace to attach them to, and nothing in the codebase starts one.

> **The 90% of Bravo's work that is the loan moving through the flow produces no trace at all.**
>
> APM sees the consoles talking to the database, and the engine talking to upstream services. It does not see the workflow.

HERE THE COMPARISON WITH LORA RUNS THE OTHER WAY

LORA's Temporal SDK emits **one trace per attempt at each activity**, carrying `@temporalWorkflowID` (which is the loan id), `resource_name` (the activity), `@error.type` and the attempt number. That is how LORA found **47 stuck loans in a 7-day window, with no code change at all**. Bravo has nothing equivalent — so if it had a class of stuck workflows, this is not how anyone would find them.

The logs are not thin. You just cannot connect them to a loan.

545,179	525,181	45,775	1.12M
INFO LINES / 7 D	ERROR LINES / 7 D	WARN LINES / 7 D	TOTAL LINES / 7 D

47% of this service's log volume is ERROR. That is not a measure of how reliable it is. It is *a side effect of how the logs are formatted*, and both halves of that matter. Clustering the error stream gives 124 patterns, and the biggest ones are individual Java stack-trace frames written as separate lines: `at <pkg>.<class>.<method>` 1,571 times, `at org.springframework.…` 874 times, `at java.base/…` 66 times. A single exception produces dozens of ERROR lines. Worse, **one Camunda job failure produces two lines at two different severities** — `ENGINE-14006` at `warn`, then `ENGINE-16004` at `error`, for the same failure.

WHAT THAT COSTS

The error stream **cannot be counted**, and every log-based monitor built on it inherits that problem.

BUT THE ENGINE'S OWN LINES ARE RICHLY TAGGED

```
message: ENGINE-16004 Exception while closing command
  context: Error while invoking Ali Cloud for PD Model
  Check Response
status:  error
version: v2.93.53
@activityId:           Activity_PD_Model_Check_NTB
@activityName:         PD Model Check NTB
@processDefinitionId: NDF2W:127:542b74c3-ab85-11f1…
@processInstanceId:    5b477587-acd4-11f1-be7a-5a673f41…
```

ONE THING HERE IS GENUINELY BETTER THAN LORA

`@processDefinitionId` carries **the process name and the exact version deployed** (`NDF2W:127`). That is better than LORA's task-queue stand-in. The Camunda job executor writes this context onto its own lines.

Failures ranked by workflow step. It took one query.

BPMN STEP	STATUS	LINES / 2 D
One Obligor Check	error	5,583
PD Model Check NTB	error	1,394
KYC Check	warn	1,377
Publish User Information (Register Keycloak)	error	789
PD Model Check NTB	warn	740
Pilot Branch Check	warn	597
One Obligor	error	434
Retrive Pefindo Spouse Profile Activity	error	326
Request Go Live	error	252
Retrive Pefindo Profile Activity	error	244
Get Agreement Number	error	150

Grouped by `@activityName`, `env:prod`, over 48 hours. About 4,750 step-level failures a day, and most of them are in one check.

> **“Activity logs are thin” is refuted. “Nobody is looking” is not.**

TWO REAL DEFECTS THE TAGGING DOES HAVE

ONLY ABOUT 5% OF LINES SAY WHICH WORKFLOW THEY CAME FROM

17,333 lines in 48 hours match `@processInstanceId:*`, out of roughly 319,000 in total. Everything from the REST layer — the consoles, the operator endpoints, every stack trace — has no workflow context at all.

NO LOG LINE CARRIES THE APPLICATION OR LEAD ID

We looked for `@applicationId` and it is **not there**. Bravo can tell you *One Obligor Check is failing 2,800 times a day*. It cannot tell you *which 400 customers are affected* without leaving Datadog. And it cannot tell you whether that is 2,800 loans failing once or forty loans failing seventy times each, because the `CARDINALITY(trace_id)` test has no equivalent here. There are no traces on the engine path.

QUESTION	BRAVO	LORA
Which step fails most?	yes — `@activityName`	yes — `@ActivityType`
On which version of the process?	yes — `@processDefinitionId`	partly — `@TaskQueue`
Which loan is this failure on?	no — you get a Camunda UUID. Resolving it needs a database lookup against `application.process_id`, and only inside the 90-day window.	yes — `@WorkflowID` *is* the application UUID
How many times has this been retried?	no — there is no attempt counter on the line	yes — `@Attempt`

Bravo can tell which upstream is failing — and this reverses the LORA finding

Outbound calls carry `@peer.hostname` for each Feign and OkHttp client, and there is **one client per upstream service**. 25 hosts come out cleanly, led by `prod-ms-user-iam` (86,066 traces in 7 days), `prod-ms-employee` (64,725), `prod-ms-product` (31,463), `prod-ms-onboarding` (29,200) and `prod-ms-master` (27,954), down to `prod-ms-backoffice` at 32.

THIS IS THE VIEW LORA DOES NOT HAVE

LORA's gateway routes about 300 inner proxies through a single endpoint (`POST /proxy/inner/request-v1`), so **21,128 errors a week all report as the same resource**. The fix, PR 1246, is merged but is not in any release tag. Bravo gets per-upstream attribution automatically, because it has **113 typed Feign clients instead of one generic proxy**. Give the point to Bravo.

THE ERRORS ARE ENTIRELY 4XX

UPSTREAM	STATUS	ERRORS / 7 D	TRACES
prod-ms-user-iam	401	**8,533**	7,964
prod-ms-repeat-order	404	5,070	3,934
prod-ms-scheduling	400	3,508	3,081
prod-ms-agreement	400	926	449
private.prod.bfi.co.id	400	671	500
gateway.bfi.co.id	422	609	610
prod-ms-kyc-proxy	400	580	474
prod-keycloak-headless	404	287	240

…and 12 more rows, all 4xx, down to 4 traces. **Not one 5xx appears in the top twenty.** Roughly 20,800 outbound errors a week, every one of them caused by the caller.

"UPSTREAM BFI SERVICES ARE UNRELIABLE" IS NOT WHAT THE DATA SAYS

What it says is that **Bravo asks upstream services for things they will not give it.** A `401` from the IAM permission endpoint, about 1,200 a day, is an authentication or token-lifetime defect *in Bravo*. The log stream confirms it: `FeignException$Unauthorized … [GET] /permission/assigned … "code":"INVALID_KEY"`, alongside 176 `TokenExpiredException`s. A `404` from repeat-order, about 720 a day, is a lookup for something that does not exist.

ERRORS PER TRACE SHOW THIS IS SPREAD WIDE, NOT STUCK IN RETRY LOOPS

IAM shows 8,533 errors across 7,964 traces — **1.07 errors per trace**, meaning each affected request fails once. LORA's equivalent table had rows showing **10,442 errors inside just 2 traces**.

Bravo fails as many loans failing once. LORA failed as two loans failing ten thousand times.

Both are worth fixing, and they need completely different fixes. Bravo's is a defect affecting a wide population. LORA's was a retry policy. This is the clearest illustration in the pack of what "bounded retry" buys and what it costs.

38 monitors, and none of them watches the workflow

CATEGORY	COUNT	EXAMPLES
Container, pod or JVM resources	11	8 pod and container CPU and memory monitors, `jvm.heap_memory`, `OutOfMemoryError: Metaspace`, faulty-deployment events
HTTP endpoints	11	success rate, 2 error-rate monitors, p90 and average latency, Apdex, throughput anomaly, `Latency Get > 500ms`
RabbitMQ failed declarations	6	one per environment: dev, sit, uat, prod, and three Sharia variants
SLO error budget	3	7-day, 30-day and 90-day — all on the same SLO
Business and upstream log alerts	5	Check Pefindo Error, Negativelist, `BranchAPIClient` error, CNV success rate, Palav callback
Test or scratch	1	`TEST- succes rate bpm`
Watching the workflow	**0**	—

Nothing watches a Camunda incident, a stuck workflow, the job-executor backlog, growth in `application_error_tracking`, or the failure rate of a step — even though §3 shows the last of those is one query away.

THREE PROBLEMS, EACH WITH AN EXACT MATCH IN LORA

EIGHT MONITORS ARE IN `ALERT` RIGHT NOW

Pod CPU and memory for Squad S&U, both latency monitors, all three SLO error-budget monitors, and the scratch `TEST-` monitor. **A monitor that has been red long enough to feel normal is not a monitor.** LORA found the same thing: a stuck-loan alert firing continuously into a chat room since January.

SIX MONITORS READ `NO DATA` AND CANNOT FIRE

All four `prod-sharia-bpm` resource monitors filter on `service:prod-sharia-bpm`, while the service is actually registered as **`prod-sharia-bpm-sharia`**. Two more formula monitors have never evaluated at all. This is exactly LORA's eight never-evaluated worker monitors, *tag mismatch and all.*

THE ONE BUSINESS MONITOR FILTERS OUT THE ENGINE'S OWN ERROR CLASS

```
logs("service:prod-ms-bpm checkpefindov2
  -status:(warn OR info) -\"ENGINE-16004\"")
  .rollup("count").by("service").last("5m") > 1
```

`ENGINE-16004` is the line that carries `@activityName` and the actual reason for the failure. Somebody excluded it, presumably because it was noisy. **The noise was the signal.** LORA's dead monitor excluded its own worst failure class for the same reason. Two teams, two platforms, the same reflex.

Dashboards. No dashboard in the org mentions `bpm` or `camunda`. Bravo LOS is covered by nine copies of a `LOS Weekly Report Latency` dashboard and two success-rate views, all of them per-service latency and 5xx. The view of the workflow itself is Camunda Cockpit, which is set to `permitAll()` in Spring Security and is not part of any on-call process recorded here.

The sharpest test of all this: five weeks, and nothing reported it

§5 shows **38 monitors matching `bpm`, and none watching the workflow.** There is a concrete test of what that costs. Between **2026-05-23 and 2026-06-30**, across **7 sessions over five weeks**, 61 process definitions designed to run arbitrary code were deployed into the production engine. On **2026-06-11 at 07:08:51**, one of them ran and captured the pod hostname.

WHAT COULD HAVE REPORTED IT	WHAT IT HELD
The 38 `bpm` monitors	**nothing** — none of them watches the workflow
Dashboards	**nothing** — none mentions `bpm` or `camunda`
APM traces on the engine path	**nothing** — there are none at all
Camunda's own `act_hi_op_log`	**zero entries** for all 61 deployments
`start_user_id_` on the instance that ran	**null**

Found by a person reading the database **three months later**, for an unrelated piece of workflow analysis.

> **The finding is not that Bravo was attacked. It is that the monitoring contributed nothing to detecting a five-week intrusion in the core service**, and that it was found by accident.

AND THIS CUTS THE OTHER WAY TOO

The engine's history tables are **what made the investigation possible.** `act_re_procdef`, `act_re_deployment`, `act_ge_bytearray` and `act_hi_varinst` kept the payloads, the timestamps, the shape of each session **and the captured command output** — three months later, with no APM and no monitoring. That is the durable-record argument working: **poor at alerting, unusually good at reconstructing what happened.** Do not replace what keeps the record. Put a monitor in front of it.

TWO MONITORS CLOSE THIS, AND THEY ARE THE CHEAPEST THINGS HERE

One on new rows in `act_re_deployment` whose `name_` falls outside the deployment naming convention. One on workflow starts whose `proc_def_key_` is outside the known set of keys. Both are single SQL conditions over tables that already exist. **Either would have fired on 2026-05-23.**

This is **not** an argument that LORA would have caught it. Temporal Cloud is a different deployment model, and no equivalent test exists there. This is Bravo's monitoring measured against Bravo's own risk.

§6 / THE LARGEST LIVE DEFECT IN THE PACK

The flood of 404s that nobody is watching

CONSOLE	SESSIONS	VIEWS	ERRORS	ERRORS PER VIEW
Surveyor Platform Prod	79,090	953,115	**1,962,909**	**2.06**
Operation Platform Prod	17,135	96,707	287,646	**2.97**
Underwriting Platform Prod	15,628	205,666	124,422	0.60
Customer Platform DF	2,594	31,412	38,221	1.22
Total, 7 days	**114,447**	**1,286,900**	**2,413,198**	**1.88**

All four consoles send browser monitoring at `rum_event_processing_state: ALL`. For scale: LORA's back office produces 416,201 errors a week, about 2.8 per view. Bravo's four consoles produce **5.8 times** that volume, at a similar rate per view. **Neither team was reading it.**

THE 404S COME FROM FOUR ENDPOINTS, AND THEY HIT MOST SESSIONS

ENDPOINT	404S / 7 D	SESSIONS
/bpm/v1/partnership-configuration/feature-configuration	**266,767**	**54,350**
/bpm/v2/surveyor-assignment-form-tab/assignment/{id}	157,623	37,359
/bpm/v1/application-document-tracking	154,097	45,916
/bpm/v1/surveyor-assignment-manual-kyc/scoring/assignment/{id}	143,180	31,565
/surveyor/assignment-detail/{id}	21,125	14,754
/surveyor/assignment	14,565	11,287

69%

OF SURVEYOR SESSIONS — 54,350 OF 79,090 — GET A 404 FROM `FEATURE-CONFIGURATION`

That is not a retry loop, and it is not an edge case. **It is what using the Surveyor Platform normally looks like.** And this is the busiest endpoint in the service: `PartnershipConfigurationController.getByApplicationId` is the top `spring.handler` resource, at **73,056 calls in 48 hours**.

WHY NO ALERT FIRED

```
(hits{service:prod-ms-bpm}
 - hits{service:prod-ms-bpm, http.status_code:5*})
  / hits{service:prod-ms-bpm} < 99
```

By that definition, a 404 is a success. Bravo's idea of health is "anything that is not a 5xx is fine". That is the same kind of assumption as treating a running workflow as a healthy workflow, and it is wrong in the same way: the system's main failure mode is invisible to the thing meant to detect failure.

WHAT THIS DOCUMENT CANNOT SAY

Whether these 404s are **harmless** — an optional setting checked on every page, where 404 just means "no override" — or a **defect**, cannot be decided from monitoring data. Somebody has to read the controller. But at 266,767 a week, across two-thirds of sessions, on the console that generates 28% of Bravo's support tickets, **somebody has to look**. And the location failures on the same console, 20,690 a week, are the kind of thing that stops a survey being submitted at all.

Where each platform keeps a lasting record

The thesis we were given was *"logs currently serve as the only durable substrate."* For Bravo that is not just wrong. **It is the exact reverse.**

LAYER	BRAVO	LORA
Business state	PostgreSQL, 238 entities, 271 tables, kept indefinitely	An ArangoDB document plus Temporal history
Status history	`application_status_log`, written by a database trigger — permanent	An append-only chain of field versions inside the document
Earlier attempts	`Application` rows chained by `prevApplication` and `currentIndex` — every reprocess attempt is a permanent row you can query	Rewinding invalidates fields; the earlier attempt has to be rebuilt from Temporal history
Step-by-step trace	Camunda `act_hi_*`, deleted after 90 days	Temporal event history, kept 30 days after a workflow closes
Traces	none for the workflow layer	30 days, one per attempt at each activity
Logs	about 1.1M lines a week, 47% errors, no loan id	about 7 days, but they carry `@WorkflowID` and `@Attempt`

WHAT BRAVO KEEPS PERMANENTLY IS ITS DATABASE

And it is **the best record either platform has** for answering "what happened to this loan?" Every reprocess attempt is a row. Nothing has to be replayed. This is the advantage the relational model genuinely delivers, and it is why this pack prefers Bravo's crude "new row per attempt" to LORA's more principled rollback.

Bravo's business facts last. Its workflow facts do not.

On day 91 you can still say what status a loan reached and when. You cannot say which step failed, how often, or why. LORA is the mirror image: the loan document and its version chain last, but the question across the whole book — "which loans are stuck in survey?" — cannot be asked at all.

Ten actions, ordered by value against effort

1 **Look at the `feature-configuration` 404** · S, DO THIS FIRST

266,767 a week, across 69% of surveyor sessions, on the busiest endpoint in the service, on the console that generates 28% of Bravo's tickets. Read the controller. Decide whether 404 is the intended "no override" response, and if it is, stop the client treating it as an error. If it is not, this is a live production defect, invisible because nothing alerts below 5xx.

2 **Make the success-rate monitors count 4xx as well** · S

Every `prod-ms-bpm` health monitor only looks for 5xx. Add monitors for 4xx per endpoint — at a minimum, `401` from IAM (8,533 a week), `404` on the four surveyor endpoints, and `400` on scheduling.

Bravo does not fail with 5xx. It fails with 4xx.

3 **Build the per-step failure monitor that already works** · S

Grouping by `@activityName` works today and nobody uses it. A monitor grouped by `@activityName` and `@processDefinitionId` is a five-minute change, and today it would read `One Obligor Check` at about 2,800 a day. Give it an owner and a runbook entry *before* creating it.

4 **Stop excluding `ENGINE-16004` from the Pefindo monitor** · S

It is the line that carries the step name and the reason for the failure. If the monitor is too noisy without the exclusion, the answer is a threshold or a group-by per step — not throwing away the informative half of the signal.

5 **Put the application id on the engine's log lines** · S–M

One extra field on the job-executor path turns a ranking of failing steps into per-loan triage, and closes the biggest gap in §3. `application.process_id` already links the two. The log line just needs the business key on it.

6 **Fix the six `No Data` monitors** · S

Four Sharia monitors filter on `service:prod-sharia-bpm`, against a service registered as `prod-sharia-bpm-sharia`. Retag them or delete them. Two formula monitors have never evaluated.

7 **Triage the eight monitors stuck in `Alert`** · S

Either the thresholds are wrong or the service is degraded. Both need an answer.

A permanently red monitor teaches people to ignore the channel.

8 **Trace the job executor** · M

A `@WithSpan` on `BaseActivity.execute()`, or the Camunda OTel plugin, gives one trace per service task, with the process name, version, step and instance.

This is the single change that would make Bravo's workflow visible in APM at all

— and it also gives latency per step, which no metric reports today.

9 **Write the stuck-loan SQL down** · S

Bravo's real advantage — you can ask questions across all loans in SQL — is unusable at 02:00 because nobody saved the query. A notebook, plus a monitor on the count, turns a capability into an operation.

10 **Read the browser monitoring in the weekly review** · S

Four consoles, 2.4 million errors a week, monitored all along and never looked at. Set an errors-per-view SLO for each console. The Surveyor Platform is at 2.06 today, and nobody could tell you if that got worse.

DO NOT CREATE ANY OF THESE ALERTS WITHOUT AN OWNER AND A RUNBOOK ENTRY

§5 shows Bravo already has 38 monitors: 8 permanently firing, 6 that cannot fire, and one that had its most useful error class filtered out. **Adding a ninth red light to the same chat room recreates the problem this document is describing.**

30, 60, 90 days

Ten recommendations, plus the runbook that has to come first, phased against **about 10 person-days a month** of Platform and SRE time, plus a slice of Squad S&U.

THE RULE FOR ORDERING THIS WORK

Repair the alert channel before adding anything to it. So recommendations 6 and 7 land in the first 30 days, and recommendation 2 does *not* — even though it is the more valuable signal. It arrives in month 2, once the channel is worth alerting into and the runbook table exists. The one exception is recommendation 3, created in phase 1 **with its owner and runbook entry attached**, because it needs no new plumbing and is the first monitor Bravo will ever have that watches the workflow.

SHARED WITH OTHER DOCUMENTS

Recommendation 1 is also rec 1 in **bravo-delivery.md** and rec 5 in **bravo-cost.md**. Recommendation 2 is also rec 3 in **bravo-testing.md**. Recommendation 10 goes with rec 3 in **bravo-delivery.md**. Phased the same way everywhere, so do them once.

Days 0–30

ONE LIVE DEFECT, AND AN ALERT CHANNEL THAT WORKS

1 **Investigate the `feature-configuration` 404.** *S&U · 2 days.* A written answer, so that 266,767 a week across 69% of sessions is explained.

1 **Fix it** — resolve the lookup, or stop the client raising the 404 as an error. *S&U · 3 days.* Surveyor error volume falls measurably in the browser monitoring.

7 **Triage the eight monitors in `Alert`.** *Platform and SRE · 2 days.* Each one fixed, re-thresholded or deleted. **No permanently red monitors left.**

6 **Fix the six `No Data` monitors.** *Platform and SRE · 1 day.*

4 **Stop excluding `ENGINE-16004`** from the Pefindo monitor. *Platform and SRE · half a day.*

3 **Build the per-step failure monitor**, with an owner and a runbook entry. *Platform and SRE · 1 day.* **Bravo's first alert on the workflow itself.**

5 **Put the application id on the engine's log lines.** *S&U · 3 days.* This unblocks per-loan triage and makes recommendations 2 and 8 far more useful.

9 **Write the stuck-loan SQL down** as a saved notebook. *S&U · 2 days.*

Days 31–60

SEE THE WORKFLOW, THEN ALERT ON IT

8 **Trace the job executor.** *S&U · 8 days.* One trace per service task. The 90% of Bravo's work that is the workflow becomes visible in APM, with latency per step and a `CARDINALITY(trace_id)` test for retry loops.

– **Build the alert runbook table** — a severity, a runbook entry and an owner for every signal, applied to the surviving old monitors too. *Platform and SRE · 2 days.* **Nothing below this row starts until it exists.**

2 **Add 4xx monitors per endpoint.** *Platform and SRE · 3 days.* Bravo's actual failure mode can be alerted on for the first time.

10 **Browser monitoring in the weekly review**, with an errors-per-view SLO for each console. *Platform, SRE and EM · 2 days.*

Days 61–90

CONSOLIDATE — NO NEW WORK

– This phase is spent on the deploy-safety and test items in **bravo-testing.md**, which use this document's output: the job-executor traces from recommendation 8 are what make the two end-to-end workflow tests *diagnosable* when they fail.

✓ **What to check at day 90:** that the monitors created in phases 1 and 2 have fired, been acted on, and are **still not muted**. The failure this document describes is not missing tooling. It is an alert that fires with nobody owning it. Ninety days is long enough to tell whether that changed.

What changes when this is done

RECOMMENDATION	EXAMPLE WHEN DONE
4xx counted as failure	A flood of 404s on the busiest endpoint pages somebody within five minutes, instead of running for months at 266,000 a week.
Per-step monitor	“One Obligor Check is failing 2,800 times a day” is a number on a dashboard, not a query somebody happened to run.
Application id on the log lines	On-call goes from a loan number to that loan’s failures in one Datadog query, the way LORA’s on-call already can.
Job-executor traces	The 90% of Bravo’s work that is the workflow becomes visible in APM, with latency per step and a `CARDINALITY(trace_id)` test that tells a wide defect apart from a retry loop.
Stuck-loan query saved	Bravo’s genuine architectural advantage — SQL over relational state — becomes something the on-call rota uses, rather than something the architecture merely allows.
Browser monitoring in the weekly review	A regression on the surveyor console is caught by a number, instead of by 315 `Release reject` tickets a month.
Monitors triaged	The alert channel means something again — which has to happen before anything else on this list is worth doing.

> **Most of the data is already there. What is missing is an alert that looks at something other than the container and the 5xx.**